Phase 1 — Capability Inventory: What Can OpenClaw Clients DO?

Analysis date: 2026-02-26 **MCP version**: ichor-mcp v0.1.0 **Source of truth**: `mcp/` module, `mcp/internal/tools/`, `mcp/internal/client/ichor.go`

---

1. Totals (Actual vs. Plan Estimate)

Artifact	Plan Estimate	Actual Count
Tools	33	**39**
Resources	7	**7** (5 static + 2 templates)
Prompts	3	**3**

The plan underestimated tools by 6. The additions are 4 discovery tools added in `RegisterTablesDiscoveryTools` / `RegisterPageActionDiscoveryTools` and 2 page action read tools (`get_page_actions`, `get_page_action`).

---

2. Complete Tool Matrix (39 Tools)

Legend: **R** = Read, **W** = Write, **V** = Validate, **A** = Analyse/Advisory

Group A — Discovery Tools (8 tools)

Registered by `RegisterDiscoveryTools` → split into Workflow and Tables subgroups.

#	Tool Name	Class	REST Endpoint	Auth Level	Business Intent
1	`discover_action_types`	R	`GET /v1/workflow/action-types`	**see note**	Know what action types (17) are available before building a workflow
2	`discover_trigger_types`	R	`GET /v1/workflow/trigger-types`	**see note**	Know trigger events (on_create, on_update, on_delete)
3	`discover_entity_types`	R	`GET /v1/workflow/entity-types`	**see note**	Know which entity categories can trigger workflows
4	`discover_entities`	R	`GET /v1/workflow/entities`	**see note**	Know specific registered entities for triggers
5	`discover_page_action_types`	R	`GET /v1/config/schemas/page-action-types`	**see note**	Know button/dropdown/separator types for page UI
6	`discover_config_surfaces`	R	`GET /v1/agent/catalog`	**see note**	Enumerate all configurable surfaces with their CRUD URLs
7	`discover_field_types`	R	`GET /v1/config/form-field-types`	**see note**	Know form field types available when building forms
8	`discover_content_types`	R	`GET /v1/config/schemas/content-types`	**see note**	Know page content block types (table, form, chart, etc.)

Note: Auth level for workflow/* and config/* discovery endpoints is not `RuleAdminOnly` — these return static schema/metadata, not sensitive data. Actual auth level depends on the respective REST route's middleware (not introspection routes). Needs per-route verification for Phase 3.

Group B — Workflow Read Tools (5 tools)

Registered by `RegisterWorkflowReadTools`.

#	Tool Name	Class	REST Endpoint(s)	Auth Level	Business Intent
9	`list_workflows`	R	`GET /v1/workflow/rules`	table-RBAC	See all automation rules in the system

#	Tool Name	Class	REST Endpoint(s)	Auth Level	Business Intent
10	`get_workflow`	R	`GET /v1/workflow/rules/{id}` + `/rules/{id}/actions` + `/rules/{id}/edges`	table-RBAC	Read a complete workflow graph with computed summary
11	`explain_workflow_node`	R	Same 2 sub-calls as above + `GET /v1/workflow/action-types/{type}/schema`	table-RBAC	Understand a single action within a workflow (depth, in/out edges)
12	`explain_workflow_path`	R	Same 2 sub-calls as get_workflow	table-RBAC	Trace execution path from start or from a named node
13	`list_action_templates`	R	`GET /v1/workflow/templates`	table-RBAC	View reusable action templates for workflow construction

Notable: `get_workflow`, `explain_workflow_node`, and `explain_workflow_path` all issue **3 separate HTTP requests** per invocation (rule, actions, edges). There is no single aggregate endpoint being used.

Group C — UI Configuration Read Tools (9 tools)

Registered by `RegisterUIReadTools` → delegates to 3 sub-registrars.

Pages (3 tools)

#	Tool Name	Class	REST Endpoint	Business Intent
14	`list_pages`	R	`GET /v1/config/page-configs/all`	Enumerate all page configurations
15	`get_page_config`	R	`GET /v1/config/page-configs/id/{id}` or `/name/{name}`	Read a page config by ID or name
16	`get_page_content`	R	`GET /v1/config/page-configs/content/{id}`	Read all content blocks for a page

Page Actions (2 tools)

#	Tool Name	Class	REST Endpoint	Business Intent
17	`get_page_actions`	R	`GET /v1/config/page-configs/actions/{page_config_id}`	List all buttons/dropdowns for a page
18	`get_page_action`	R	`GET /v1/config/page-actions/{action_id}`	Read a single page action with full type details

Content Blocks (4 tools)

#	Tool Name	Class	REST Endpoint	Business Intent
19	`list_table_configs`	R	`GET /v1/data/configs/all`	Enumerate all table/widget configurations
20	`get_table_config`	R	`GET /v1/data/id/{id}` or `/name/{name}`	Read a table config with JSONB settings
21	`list_forms`	R	`GET /v1/config/forms`	Enumerate all form definitions
22	`get_form_definition`	R	`GET /v1/config/forms/{id}/full` or `/name/{name}/full`	Read a form with all its fields, types, validation

---

Group D — Search / Introspection Tools (2 tools)

Registered by `RegisterSearchTools`.

#	Tool Name	Class	REST Endpoint(s)	Auth Level	Business Intent

#	Tool Name	Class	REST Endpoint(s)	Auth Level	Business Intent
23	`search_database_schema`	R	`GET /v1/introspection/schemas` (no args) `GET /v1/introspection/schemas/{schema}/tables` (schema only) `GET /v1/introspection/tables/{schema}/{table}/columns` + `/relationships` (schema+table)	**RuleAdminOnly**	Map the full database schema — schemas, tables, columns, foreign keys
24	`search_enums`	R	`GET /v1/introspection/enums/{schema}` (name omitted) → **RuleAdminOnly** `GET /v1/config/enums/{schema}/{name}/options` (name provided) → **RuleAny**	**Mixed**	Browse PostgreSQL enum types and their human-friendly labels

Critical observation: `search_enums` has asymmetric auth:

- Listing enum types (no `name`) calls the raw introspection endpoint → admin token required.
- Fetching enum values (with `name`) calls the combined options endpoint → any authenticated user.

This means a non-admin agent can fetch enum values but cannot list which enums exist.

Group E — Workflow Write Tools (3 tools)

Registered by `RegisterWorkflowWriteTools`.

#	Tool Name	Class	REST Endpoint	Default Behavior	Business Intent
25	`validate_workflow`	V	`POST /v1/workflow/rules/full?dry_run=true`	Always dry-run	Check a workflow graph for errors before committing
26	`create_workflow`	W	`POST /v1/workflow/rules/full`	Pre-validates by default; `validate=false` skips	Create a new automation rule with full action graph
27	`update_workflow`	W	`PUT /v1/workflow/rules/{id}/full`	Pre-validates by default; `validate=false` skips	Replace an existing workflow's rule + action graph

Validate-first pattern: Both `create_workflow` and `update_workflow` auto-run `validate_workflow` before committing unless the caller explicitly passes `validate: false`.

Group F — UI Configuration Write Tools (8 tools)

Registered by `RegisterUIWriteTools` → delegates to pages and content blocks.

Pages Write (4 tools)

#	Tool Name	Class	REST Endpoint	Business Intent
28	`create_page_config`	W	`POST /v1/config/page-configs`	Create a new top-level page container
29	`update_page_config`	W	`PUT /v1/config/page-configs/id/{id}`	Modify an existing page's metadata
30	`create_page_content`	W	`POST /v1/config/page-content`	Add a content block (table, form, chart, etc.) to a page
31	`update_page_content`	W	`PUT /v1/config/page-content/{id}`	Modify a content block's layout/label/visibility

Content Blocks Write (4 tools)

#	Tool Name	Class	REST Endpoint	Business Intent

#	Tool Name	Class	REST Endpoint	Business Intent
32	`create_form`	W	`POST /v1/config/forms`	Create a new form definition for data entry
33	`add_form_field`	W	`POST /v1/config/form-fields`	Append a field to an existing form
34	`create_table_config`	W	`POST /v1/data`	Create a new table/widget configuration
35	`update_table_config`	W	`PUT /v1/data/{id}`	Modify an existing table/widget configuration

Group G — Validation Tools (1 tool)

#	Tool Name	Class	REST Endpoint	Business Intent
36	`validate_table_config`	V	`POST /v1/data/validate`	Validate table config JSONB without saving

Group H — Analysis / Advisory Tools (3 tools)

Registered by `RegisterAnalysisTools`. These are **agent-side analytical tools** — they call the same read endpoints but add reasoning logic in the MCP server itself.

#	Tool Name	Class	REST Endpoint(s)	Business Intent
37	`analyze_workflow`	A	`GET /v1/workflow/rules/{id}` + actions + edges	Assess a workflow for complexity, gaps, and suggestions
38	`suggest_templates`	A	`GET /v1/workflow/templates/active` + `GET /v1/workflow/action-types`	Given a use-case description, suggest relevant templates/actions
39	`show_cascade`	A	`GET /v1/workflow/rules` (then filters client-side)	Show which workflows would fire when a given entity changes

Notable: `show_cascade` fetches ALL workflow rules and filters client-side inside the MCP server process. At scale, this could be a performance issue with many rules.

3. Tool Classification Summary

Classification	Count	Tools
Read (R)	23	#1–24 (minus V tools)
Write (W)	11	#26–35
Validate (V)	2	#25 (validate_workflow), #36 (validate_table_config)
Analyse (A)	3	#37–39
Total	**39**	

4. Context Mode Mapping

`--context workflow` (17 tools)

Intended persona: **Workflow automation builder / automation auditor**

Group	Tools
Workflow Discovery	discover_action_types, discover_trigger_types, discover_entity_types, discover_entities
Workflow Read	list_workflows, get_workflow, explain_workflow_node, explain_workflow_path, list_action_templates
Search	search_database_schema, search_enums
Workflow Write	validate_workflow, create_workflow, update_workflow
Analysis	analyze_workflow, suggest_templates, show_cascade

`--context tables` (24 tools)

Intended persona: **UI/UX configurator — pages, forms, tables**

Group	Tools

Group	Tools
Tables Discovery	discover_page_action_types, discover_config_surfaces, discover_field_types, discover_content_types
UI Read — Pages	list_pages, get_page_config, get_page_content
UI Read — Page Actions	get_page_actions, get_page_action
UI Read — Content Blocks	list_table_configs, get_table_config, list_forms, get_form_definition
Search	search_database_schema, search_enums
UI Write — Pages	create_page_config, update_page_config, create_page_content, update_page_content
UI Write — Content Blocks	create_form, add_form_field, create_table_config, update_table_config
Validation	validate_table_config

`--context all` (39 tools) — **default**

All tools from both modes. An agent running `--context all` can simultaneously build workflow automation AND configure UI pages/forms/tables.

---

5. Resource Catalog (7 Resources)

Static Resources (5)

URI	Name	Data Surfaced	Auth Gate
`config://catalog`	Config Surface Catalog	All configurable surfaces in Ichor with CRUD endpoint URLs, discovery links, constraints	**REST auth**
`config://action-types`	Workflow Action Types	All 17 workflow action types with JSON config schemas, output ports, categories	**REST auth**
`config://field-types`	Form Field Types	All form field types with JSON config schemas	**REST auth**
`config://table-config-schema`	Table Config JSON Schema	JSON schema for `table_configs.config` JSONB column	**REST auth**
`config://layout-schema`	Layout Config JSON Schema	JSON schema for `page_content.layout` JSONB column	**REST auth**

Resource Templates (2)

URI Template	Name	Data Surfaced	Auth Gate
`config://db/{schema}/{table}`	Database Table Schema	Columns (name, type, nullable, default) for any table	**RuleAdminOnly**
`config://enums/{schema}/{name}`	Enum Options	Enum values with human-friendly labels	**RuleAny**

Observations:

- `config://db/{schema}/{table}` calls the `RuleAdminOnly` introspection endpoint — this template is effectively admin-only, though there is no validation preventing non-admin clients from requesting it (they will receive a 403 from the backend).
- `config://enums/{schema}/{name}` calls the combined options endpoint (RuleAny) — available to all authenticated users.
- There are no resources that surface runtime/data-plane data (no orders, inventory, users, etc.).
- The `config://layout-schema` and `config://table-config-schema` are passive JSON Schema documents — they help agents generate valid payloads without making round-trips.

---

6. Prompt Catalog (3 Prompts)

Prompt 1: `build-workflow`

Attribute	Detail
Arguments	`trigger` (required), `entity` (required)
Pre-fetched context	All trigger types + all action types (2 API calls)

Attribute	Detail
Guided flow	1. Confirm trigger/entity validity 2. Design action graph 3. Choose action types/configs 4. Define edges 5. validate_workflow 6. create_workflow
Useful for	End-to-end workflow construction with type-aware guidance

Observation: The prompt front-loads **all 17 action types** (including destructive ones like `allocate_inventory`, `send_email`, `commit_allocation`) into agent context before the agent has stated what it wants to do. The agent immediately knows all available automation capabilities — including ones irrelevant or dangerous to its stated task.

Prompt 2: `configure-page`

Attribute	Detail
Arguments	`entity` (required)
Pre-fetched context	Content types + field types (2 API calls)
Guided flow	1. Create page config 2. Add content blocks 3. Configure layouts 4. Reference tables/forms
Useful for	Building responsive page layouts with grid/tab organization

Prompt 3: `design-form`

Attribute	Detail
Arguments	`entity` (required)
Pre-fetched context	Field types only (1 API call)
Guided flow	1. Explore DB schema 2. Create form 3. Add fields per column type 4. Configure validation
Useful for	Schema-driven form generation mapping DB columns to form fields

Observation: The `design-form` prompt explicitly instructs the agent to call `search_database_schema` as step 1 — meaning the prompt assumes the agent has admin-level access to run schema introspection.

---

7. Business Domain Coverage Matrix

For each Ichor business domain, how much CRUD does MCP expose?

Domain	Create	Read	Update	Delete	Coverage	Notes
Workflow Rules	✅	✅	✅	❌	75%	No DELETE tool
Workflow Actions	✅ (via full-rule create)	✅	✅ (via full-rule update)	❌	75%	Actions managed as part of full rule payload
Workflow Edges	✅ (via full-rule)	✅	✅ (via full-rule)	❌	75%	Same as actions
Action Templates	❌	✅	❌	❌	25%	Read-only; no creation or management
Page Configs	✅	✅	✅	❌	75%	No DELETE tool
Page Content	✅	✅	✅	❌	75%	No DELETE tool
Page Actions	❌	✅	❌	❌	25%	Read-only; no create/update/delete
Forms	✅	✅	❌	❌	50%	No UPDATE form tool (client has `UpdateForm` but not exposed)
Form Fields	✅	✅ (via get_form_definition)	❌	❌	50%	No UPDATE/DELETE field (client has `UpdateFormField` but not exposed)
Table Configs	✅	✅	✅	❌	75%	No DELETE tool

Domain	Create	Read	Update	Delete	Coverage	Notes
DB Schema	❌	✅ (admin)	❌	❌	R-only	Introspection only, admin-gated
Enum Types	❌	✅ (partial)	❌	❌	R-only	Listing enums is admin-only; reading values is any-user
Business Data	❌	❌	❌	❌	0%	Orders, inventory, products, users — completely inaccessible
Workflow Execution	❌	❌	❌	❌	0%	No execution history, status, or runtime state
Users / Roles	❌	❌	❌	❌	0%	Not in MCP scope
Alerts	❌	❌	❌	❌	0%	No alert configuration tools

Summary: MCP covers the **configuration plane** (workflows, UI) at ~50–75% CRUD completeness. The **data plane** (business records, runtime state) is entirely absent.

---

8. Configuration-Plane vs. Data-Plane Access

Plane	Access	Examples
Configuration plane	✅ Partial (read+write, no delete)	Workflow rules, page configs, forms, table configs
Data plane	❌ None	Orders, inventory movements, product records, user accounts
Runtime plane	❌ None	Workflow execution history, job status, trigger logs
Schema plane	✅ Admin-only	Database table structure, column definitions, enum values

---

9. Achievable End-to-End Enterprise Use Cases

The following workflows can be completed entirely via MCP tools today:

UC-1: Create a New Inventory Approval Workflow

Prerequisites: Admin token (for schema introspection in design phase), valid entity registered **Steps**:

1. `discover_action_types` — find `seek_approval`, `evaluate_condition`, `send_email`
2. `discover_trigger_types` + `discover_entities` — find `on_create` for `inventory_adjustments`
3. `search_database_schema` — inspect inventory schema to understand field names (admin required)
4. `suggest_templates` (use_case: "approve inventory adjustments") — get template suggestions
5. `validate_workflow` — dry-run the graph
6. `create_workflow` — commit the automation rule
7. `analyze_workflow` — post-creation quality check

UC-2: Build a Form for a New Product Page

Prerequisites: Standard user token with config write permissions **Steps**:

1. Invoke `design-form` prompt with `entity=products`
2. `search_database_schema` (admin required) — map product columns to field types
3. `create_form` — create the form definition
4. `add_form_field` (× N) — add one field per relevant column
5. `create_page_config` — create the page container
6. `create_page_content` — add the form as a content block on the page

UC-3: Audit Automation Impact Before a Schema Change

Steps:

1. `show_cascade` — see which workflows trigger on the target entity
2. `get_workflow` (× N) — read each triggering workflow
3. `explain_workflow_path` — trace critical paths
4. `analyze_workflow` — get complexity/risk assessment

UC-4: Configure a Data Table View for Orders

Steps:

1. `discover_content_types` + `discover_config_surfaces` — understand structure
2. `search_database_schema` (admin) — get orders table columns
3. `validate_table_config` — validate the JSONB config
4. `create_table_config` — save the widget config
5. `create_page_config` + `create_page_content` — embed the table on a page

What Cannot Be Done End-to-End Today:

- **Cannot delete anything** — no cleanup or lifecycle management
- **Cannot verify a workflow executed** — no runtime state access
- **Cannot manage users or permissions** — zero identity tools
- **Cannot read actual business data** — no orders, products, inventory
- **Cannot trigger a workflow manually** — fire/replay not exposed
- **Cannot update an existing form's fields** — only add_form_field (no update/delete)
- **Cannot manage page actions** — read only
- **Cannot manage action templates** — read only

---

10. Client HTTP Capabilities Not Exposed as MCP Tools

The `client/ichor.go` HTTP client defines methods that **exist but have no corresponding MCP tool**. These represent intentional or accidental under-exposure:

Client Method	REST Endpoint	Missing Tool Implication
`UpdateForm`	`PUT /v1/config/forms/{form_id}`	Cannot update a form's metadata after creation
`UpdateFormField`	`PUT /v1/config/form-fields/{field_id}`	Cannot modify a field after it's been added
`GetActiveTemplates`	`GET /v1/workflow/templates/active`	Used internally by `suggest_templates` but not directly addressable

---

11. Key Structural Observations

1. **Thin translation layer**: The MCP server is a pure HTTP-client wrapper. All authorization is delegated entirely to the Ichor REST API backend. The MCP layer does zero per-tool authorization checks.

2. **Passthrough JSON**: All write tools pass opaque `json.RawMessage` payloads directly to the REST API. The MCP server performs no schema validation, type checking, or injection sanitization before forwarding.

3. **Validate-first pattern (workflow only)**: Workflow writes have a built-in validate-first guard. UI writes (`create_page_config` , `create_form` , etc.) have no equivalent pre-validation step except `validate_table_config` .

4. **No DELETE surface at all**: Not a single MCP tool calls DELETE on any endpoint. This is consistent across all domains.

5. **Graph traversal without cycle guards**: `explain_workflow_path` uses a `visited` map to detect cycles (prints `↻ (continues above)`) — so infinite loops won't hang the process. However, depth is unbounded — a very deep graph will fully traverse.

6. **Three-call pattern**: Workflow read tools fetch rule + actions + edges as three separate HTTP requests. No batching, no streaming. Each `get_workflow` call makes 3 REST round-trips to the backend.

7. **Context mode has no enforcement mechanism**: The context mode is applied only at startup during tool registration. Once the server is running with `--context all` , there is no runtime way to restrict tool access per-request. A single token grants all registered tools.

Output for Phase 2 (Access Boundary Analysis) → `PHASE2_ACCESS_BOUNDARY.md`